002 - GitHub CI/CD Deployment

Current Flow: You manually run `mprocs-deploy.yaml` on your machine -> VPS updates. **New Flow:** You `git push` -> GitHub Actions builds the images -> Watchtower on your VPS automatically pulls and updates them.

Here is your complete workflow setup.

1. Create the Workflow File

Create (or update) the file in your project at: `.github/workflows/deploy.yml`

```yaml
name: Build and Push Docker Images

on:
  push:
    branches: ["main"]
    # Only trigger if relevant files change
    paths:
      - "apps/**"
      - "services/**"
      - "contracts/**"
      - "packages/**"
      - "docker-compose.yml"
      - "docker-compose.prod.yml"
      - "gateway/**"
      - ".github/workflows/deploy.yml"
  workflow_dispatch: # Allows manual triggering from the GitHub UI

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

env:
  REGISTRY: ghcr.io
  # Automatically gets "org-name/repo-name" (e.g., proxeon/bootcamp)
  IMAGE_NAME: ${{ github.repository }}

jobs:
  # ------------------------------------------------------------------
  # JOB 1: API (.NET)
  # ------------------------------------------------------------------
  api:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
```

```yaml
      - name: Filter Changes
        uses: dorny/paths-filter@v2
        id: changes
        with:
          filters: |
            src:
              - 'services/api-dotnet/**'
              - 'contracts/**'

      - name: Set up Docker Buildx
        if: steps.changes.outputs.src == 'true'
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        if: steps.changes.outputs.src == 'true'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.CR_USER }}
          password: ${{ secrets.CR_PAT }}

      - name: Build and Push API
        if: steps.changes.outputs.src == 'true'
        uses: docker/build-push-action@v5
        with:
          context: .
          file: services/api-dotnet/Dockerfile
          push: true
          platforms: linux/amd64
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/api:latest
          labels: |
            org.opencontainers.image.source=https://github.com/${{
github.repository }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # ------------------------------------------------------------------
  # JOB 2: WEB (Next.js)
  # ------------------------------------------------------------------
  web:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Filter Changes
        uses: dorny/paths-filter@v2
        id: changes
        with:
          filters: |
```

```
            src:
              - 'apps/web/**'
              - 'packages/**'
              - 'contracts/**'

      - name: Set up Docker Buildx
        if: steps.changes.outputs.src == 'true'
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        if: steps.changes.outputs.src == 'true'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.CR_USER }}
          password: ${{ secrets.CR_PAT }}

      - name: Build and Push Web
        if: steps.changes.outputs.src == 'true'
        uses: docker/build-push-action@v5
        with:
          context: .
          file: apps/web/Dockerfile
          push: true
          platforms: linux/amd64
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/web:latest
          labels: |
            org.opencontainers.image.source=https://github.com/${{
github.repository }}
          cache-from: type=gha
          cache-to: type=gha,mode=max

  # ---------------------------------------------------------------------
  # JOB 3: ADMIN (Vite)
  # ---------------------------------------------------------------------
  admin:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Filter Changes
        uses: dorny/paths-filter@v2
        id: changes
        with:
          filters: |
            src:
              - 'apps/admin/**'
              - 'packages/**'
              - 'contracts/**'
```

```
      - name: Set up Docker Buildx
        if: steps.changes.outputs.src == 'true'
        uses: docker/setup-buildx-action@v3

      - name: Log in to GHCR
        if: steps.changes.outputs.src == 'true'
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ secrets.CR_USER }}
          password: ${{ secrets.CR_PAT }}

      - name: Build and Push Admin
        if: steps.changes.outputs.src == 'true'
        uses: docker/build-push-action@v5
        with:
          context: .
          file: apps/admin/Dockerfile
          push: true
          platforms: linux/amd64
          tags: ${{ env.REGISTRY }}/${{ env.IMAGE_NAME }}/admin:latest
          labels: |
            org.opencontainers.image.source=https://github.com/${{
github.repository }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

2. Configure GitHub Secrets

For the CI/CD pipeline to push images securely, you need to add your credentials to the GitHub Repository Secrets.

1. Go to your repository on GitHub -> **Settings** -> **Secrets and variables** -> **Actions**.
2. Click **New repository secret** and add the following two secrets:

Name	Value Example	Description
`CR_USER`	`allaboutevemirolive`	The exact GitHub username that generated the PAT
`CR_PAT`	`ghp_xxxx...`	The Personal Access Token you generated in Step 1

3. Commit and Push

```
git add .github/workflows/deploy.yml
git commit -m "ci: enable automated deployments"
git push origin main
```

What will happen?

1. **Trigger:** GitHub Actions will start automatically when you push to the `main` branch.

2. **Smart Filtering:** It detects which folders were modified. If you only changed code in `apps/web`, **only** the `web` job will run. The `api` and `admin` jobs will be skipped instantly, saving you massive amounts of build time.
3. **Build & Push:** It securely logs into `ghcr.io` using your `CR_PAT`, builds the Docker images, and pushes them with the `latest` tag.
4. **Auto-Deploy:** Watchtower (running on your DigitalOcean VPS) will detect the new images within ~300 seconds, gracefully shut down the old containers, and start the new ones with zero manual intervention.